

1 Aufgabe 1

Die Messungen wurden auf einer NVIDIA GeForce RTX 4060 durchgeführt. Die verwendete GPU besitzt 24 SMs, 3072 FP32-Kerne, einen FP32-Peak von ungefähr 15,3 TFLOP/s und eine Speicherbandbreite von 272 GB/s.

1.1 Verwendete Kernel

1.1.1 FMA-Kernel für Skalierung und GPU-Warm-up

Der erste verwendete Kernel ist ein *fused multiply add (fma)* Kernel: `kernel_fma` in `fma.cu`. Der Kernel simuliert eine große arithmetische Last bei gleichzeitig wenigen Speicherzugriffen. Damit kann gemessen werden, ab welcher Problemgröße genügend Threads vorhanden sind, um die FP32-Recheneinheiten der GPU voll auszulasten.

Jeder Thread bearbeitet genau ein Element, d.h. er lädt einen 4 Byte großen Wert, führt mindestens eine FMA Operation auf diesem Wert aus und schreibt anschließend das Ergebnis zurück. Dadurch ist der Speicherbedarf minimal (4 Byte *read* und 4 Byte *write*), während die arithmetische Last über die Anzahl der Iterationen der FMA Operation mithilfe des Parameters `iters` skaliert werden kann.

```
float x = in[idx];
float acc0 = x, acc1 = x * 0.5f + 1.0f;

for (int i = 0; i < iters; ++i) {
    acc0 = fmaf(acc0, a, b);
    acc1 = fmaf(acc1, a, b);
}

out[idx] = acc0 + acc1;
```

Das Ergebnis wird in den Ausgabepuffer (`out[idx]`) geschrieben, damit der Compiler die Rechenschleife nicht entfernen kann.

1.1.2 FMA Divergenz-Kernel für unterschiedliche Kontrollflusspfade

Der zweite verwendete Kernel ist ebenfalls ein *fused multiply add (fma)* Kernel: `kernel_fma_div` in `fma.cu`. Im Unterschied zum 1. Kernel fügt dieser gezielt einen datenabhängigen Kontrollfluss hinzu. Damit kann nicht nur die reine Rechenleistung gemessen werden, sondern auch der Effekt, der entsteht, wenn verschiedene *Lanes* eines Warps unterschiedliche Pfade nehmen können.

Im Unterschied zu `kernel_fma` ist die Arbeit pro Thread in `kernel_fma_div` nicht uniform verteilt. Einige Threads nehmen einen schweren Pfad mit vielen FMA-Operationen, während andere einen leichten Pfad mit deutlich weniger Arbeit nehmen. Der Parameter `stride` steuert dabei, ob diese Pfade innerhalb eines Warps gemischt sind oder ob ganze Warps homogen bleiben.

1.1.3 Mandelbrot Kernel der das natürliche Auftreten von Divergenz simuliert

Der Kernel in `mandelbrot.cu` berechnet für jeden Pixel eine Mandelbrot-Iteration

```
z = z * z + c
```

bis entweder $|z| > 2$ gilt oder `maxIter` erreicht ist. Es handelt sich hierbei ebenfalls um ein rechenintensives Problem mit sehr wenigen Speicherzugriffen, da jeder Thread am Ende im Wesentlichen nur die erreichte Iterationszahl schreibt.

Der Unterschied zum FMA-Kernel ist, dass die Divergenz nicht künstlich über `stride` eingestellt wird. Sie entsteht natürlich aus den Daten: Pixel außerhalb der Mandelbrot-Menge brechen früh ab, Pixel nahe am Rand laufen deutlich länger, und Pixel im Inneren erreichen oft `maxIter`. Wenn solche Pixel in derselben Warp liegen, müssen die unterschiedlich langen Schleifenpfade serialisiert werden. Mandelbrot zeigt also denselben SIMT-Effekt in einem realistischeren Algorithmus als der FMA-Kernel, ist aber weniger exakt kontrollierbar als `kernel_fma_div`.

1.1.4 Monte Carlo Kernel um das zufällige Auftreten von Divergenz zu simulieren

Der Monte-Carlo Kernel in `montecarlo.cu` schätzt π , indem jeder Thread zufällige Punkte im Einheitsquadrat erzeugt und prüft, ob sie im Viertelkreis liegen:

```
if (x * x + y * y <= 1.0f)
    ++local;
```

Dabei ist die Divergenz nicht räumlich strukturiert wie bei der Berechnung der Mandelbrot Menge, sondern sie entsteht zufällig durch die Stichproben. In einer Warp nehmen die Lanes daher statistisch gemischte Pfade.

Monte Carlo ist allerdings kein isolierter Divergenz-Benchmark, da Die Laufzeit zusätzlich Kosten für die Erzeugung der Zufallszahlen, lokale Zähler und eine globale Reduktion per `atomicAdd` enthält. Deshalb dient dieser Kernel nur als ergänzender Referenzfall, um zu illustrieren, wie zufällige Kontrollflüsse in einem stochastischen GPU-Algorithmus auftreten.

1.2 Skalierung über die Problemgröße

Für die Skalierung wurde die Anzahl der Iterationen fest gewählt: `iters = 512`. Die Problemgröße n wurde von 1024 bis 134 Millionen Elementen variiert. Die Ergebnisse liegen in `outputs/scaling.csv`; die daraus erzeugte Grafik ist in Abbildung 1 dargestellt.

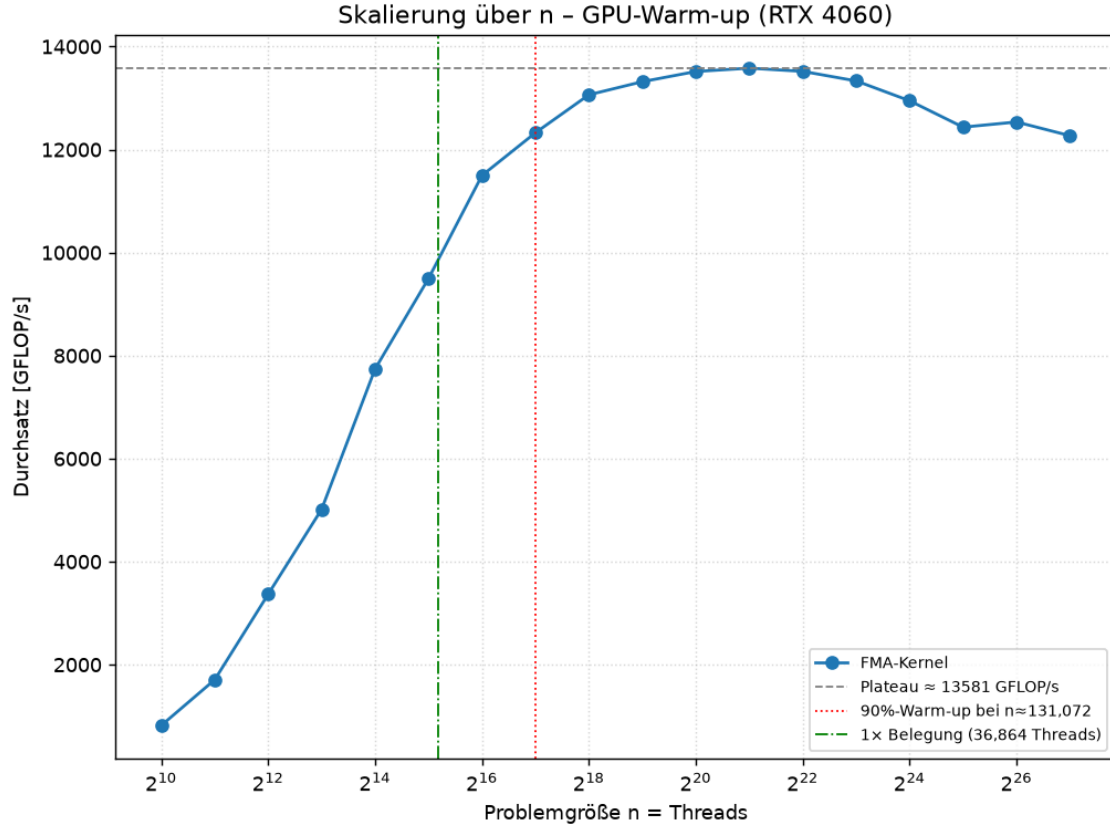


Abbildung 1: Durchsatz des FMA-Kernels in Abhängigkeit von der Problemgröße.

n = Threads	GFLOP/s	Vielfaches der Sättigung
1 024	819	0,03
16 384	7 740	0,44
32 768	9 498	0,89
65 536	11 498	1,78
131 072	12 327	3,6
262 144	13 064	7,1
1 048 576	13 515	28
16 777 216	12 947	455
67 108 864	12 536	1820

Tabelle 1: Auszug aus der Skalierungsmessung.

Die theoretische Sättigungsschwelle der verwendeten GPU liegt somit bei

$$24 \text{ SMs} \cdot 1536 \text{ Threads/SM} = 36\,864$$

gleichzeitig laufenden Threads. Diese Schwelle ist jedoch nicht identisch mit dem praktischen Plateau. In der Messung wurde die 90-Prozent-Region erst bei etwa 131 072 Threads erreicht, also bei ungefähr dem 3,6-fachen der Thread-Kapazität. Das Plateau liegt bei etwa 13,6 TFLOP/s.

Die Messung zeigt damit den typischen GPU-Warm-up: Bei kleinen Problemgrößen ist die GPU nicht ausgelastet. Erst wenn die Problemgröße groß genug ist, d.i. ausreichend viele Warps und genügend Arbeit vorhanden ist, kann der Scheduler Latenzen durch Wechseln auf andere lauffähige Warps ausgleichen. Dagegen sind die Kerne einer CPU auf niedrige Latenz, Sprungvorhersage, Out-of-Order-Ausführung und große Caches optimiert, sodaß ein einzelner Thread

bereits sinnvoll Arbeit leisten kann. Im Unterschied zur CPU muss die GPU mit sehr vielen unabhängigen Threads versorgt werden.

1.3 Gezielte Warp-Divergenz

Auch für den Divergenzversuch mit `kernel_fma_div` hat ein Thread genau ein Element bearbeitet, wobei die Rechenlast dieses Mal von der Thread-ID abhing. Dabei hat die Hälfte der Threads einen schweren Pfad mit `iters` FMA Operationen, die andere Hälfte einen leichten Pfad mit `iters/8` FMA Operationen genommen. Der Parameter `stride` hat festgelegt, wie diese Pfade auf die Lanes eines Warps verteilt wurden.

```
bool heavy = ((idx / stride) & 1) == 0;

if (heavy) {
    for (int i = 0; i < iters; ++i)
        acc = fmaf(acc, a, b);
} else {
    for (int i = 0; i < iters / 8; ++i)
        acc = fmaf(acc, a, b);
}
```

Während sich bei `stride = 1` schwere und leichte Threads innerhalb eines Warps maximal stark abwechseln, sind Warps bei *strides*, die Vielfache von 32 sind, homogen. Strides knapp oberhalb von 32 schneiden dagegen wieder durch Warp-Grenzen und erzeugen teilweise divergente Warps. Dadurch kann direkt gemessen werden, ob der Performance-Einbruch wirklich ein Intra-Warp-Effekt ist.

Die Messdaten liegen in `outputs/divergence.csv`; die daraus erzeugte Grafik ist in Abbildung 2 dargestellt.

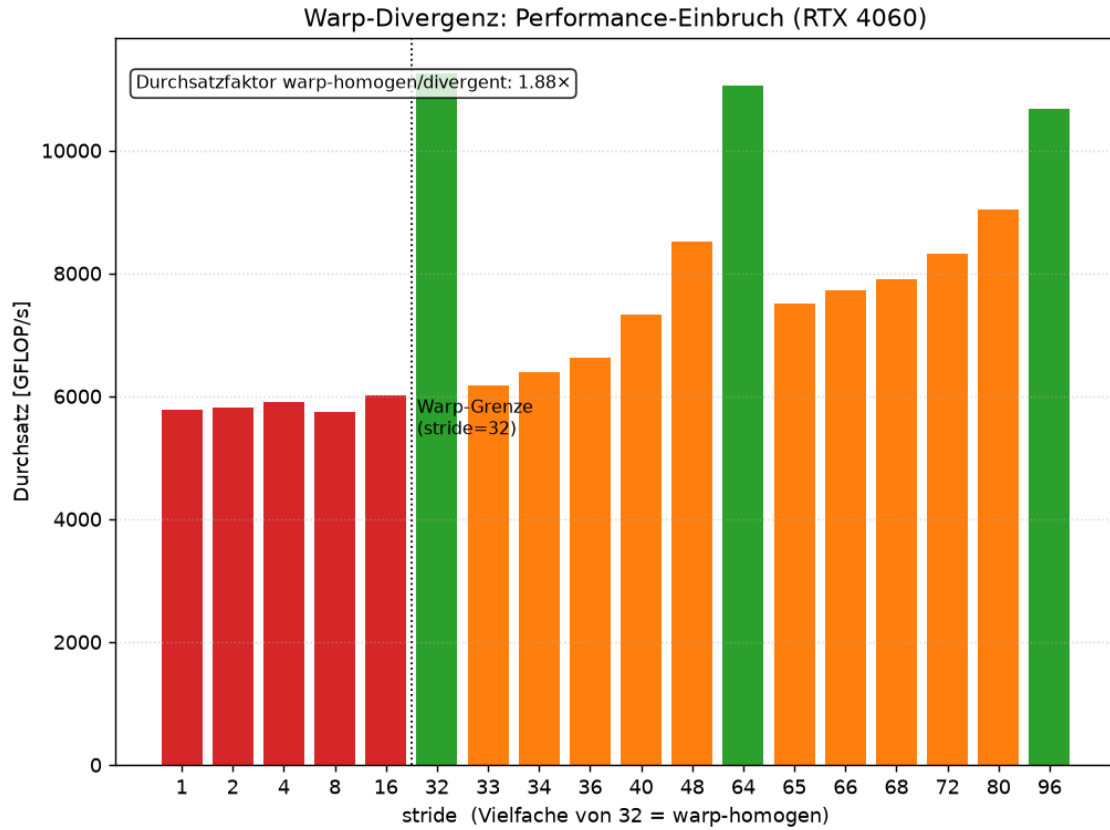


Abbildung 2: Performance-Einbruch durch Warp-Divergenz.

Stride	GFLOP/s	Zeit [ms]	Regime
1	5 778	13,38	max. divergent
2	5 822	13,28	divergent
4	5 913	13,08	divergent
8	5 739	13,47	divergent
16	6 016	12,85	divergent
32	11 264	6,86	divergenzfrei
33	6 170	12,53	teils divergent
34	6 384	12,11	teils divergent
36	6 620	11,68	teils divergent
40	7 334	10,54	teils divergent
48	8 525	9,07	teils divergent
64	11 065	6,99	divergenzfrei
65	7 511	10,29	teils divergent
66	7 732	10,00	teils divergent
68	7 901	9,78	teils divergent
72	8 315	9,30	teils divergent
80	9 039	8,55	teils divergent
96	10 684	7,24	divergenzfrei

Tabelle 2: Divergenzmessung bei gleicher Gesamtarbeit.

Die Auswertung zeigt, dass der Durchsatz genau bei *strides* springt, die Vielfache der Warp-Größe 32 sind. Für *stride* < 32 enthält jeder Warp beide Kontrollflusspfade, sodaß der Warp diese Pfade serialisieren muss. Der gemessene Durchsatz liegt in diesem Fall nur bei etwa 5,7

bis 6,0 TFLOP/s. Bei `stride = 32, 64` und `96` sind die Warps homogen; der Durchsatz steigt auf etwa 10,7 bis 11,3 TFLOP/s. Die Zwischenwerte `33` bis `48` und `65` bis `80` schneiden die Warp-Grenzen nicht sauber und sind deshalb nur teilweise divergent. Die vollständig divergente Variante ist gegenüber den warp-homogenen Fällen ungefähr um den Faktor 1,9 langsamer.

1.4 Warum Divergenz auf der GPU teuer ist

Eine GPU führt Threads in Warps aus, die in der getesteten GPU aus 32 Lanes bestehen. Im SIMT-Modell müssen alle Lanes gemeinsam einen Instruktionsstrom ausführen. Das heißt, wenn einzelne Lanes unterschiedliche Pfade eines `if/else` nehmen, kann die GPU diese Pfade nicht gleichzeitig als unabhängige Kontrollflüsse ausführen. Stattdessen wird zuerst einer der beiden Pfade mit einer Aktivitätsmaske ausgeführt, danach der andere Pfad mit der umgekehrten Maske. Lanes, die zum jeweils anderen Pfad gehören, sind in dieser Zeit inaktiv.

Dadurch addieren sich die Laufzeiten der Pfade innerhalb des Warps. Im Extremfall muss ein Warp viele verschiedene Pfade nacheinander ausführen. Genau dieser Effekt ist in Abbildung 2 sichtbar: Der Performance-Sprung liegt nicht bei beliebigen Blockgrößen, sondern exakt an den Vielfachen der Warp-Grenze von 32 Threads.

1.5 Warum dieselbe Verzweigung auf der CPU weniger stört

Eine CPU besitzt kein Warp-Modell mit 32 aneinander gekoppelten Lanes. Stattdessen führt ein CPU-Kern einen einzelnen Instruktionsstrom aus und reagiert auf Verzweigungen mit Sprungvorhersage und spekulativer Ausführung. Bei gut vorhersagbaren Mustern kostet eine Verzweigung daher fast nichts. Zudem besitzt jeder CPU-Kern jeweils einen eigenen Kontrollfluss.

Der Architekturunterschied ist deshalb grundsätzlich: Die GPU spart Steuerlogik und erzielt bei einem uniformen Kontrollfluss einen sehr hohen Durchsatz, muss dafür aber bei einem divergenten Kontrollfluss innerhalb eines Warps serialisieren. Dagegen investiert die CPU wesentlich mehr Logik pro Kern, um auch einen komplexen Kontrollfluss mit niedriger Latenz auszuführen, kann dafür allerdings nicht den Durchsatz einer GPU bei einem uniformen Kontrollfluss erreichen.

1.6 Anhang: Ergänzende Referenzkernel

Zusätzlich zu den synthetischen FMA- und Divergenz-Kernen wurden der Mandelbrot- und der Monte-Carlo-Kernel als ergänzende Referenzfälle ausgewertet.

1.6.1 Mandelbrot-Kernel

Der Mandelbrot-Kernel berechnet für jeden Pixel, nach wie vielen Iterationen die Folge $z_{n+1} = z_n^2 + c$ den Radius 2 überschreitet. Dadurch hat jeder Thread eine datenabhängige Laufzeit. Punkte außerhalb der Menge brechen früh ab, Punkte nahe am Rand oder innerhalb der Menge laufen deutlich länger.

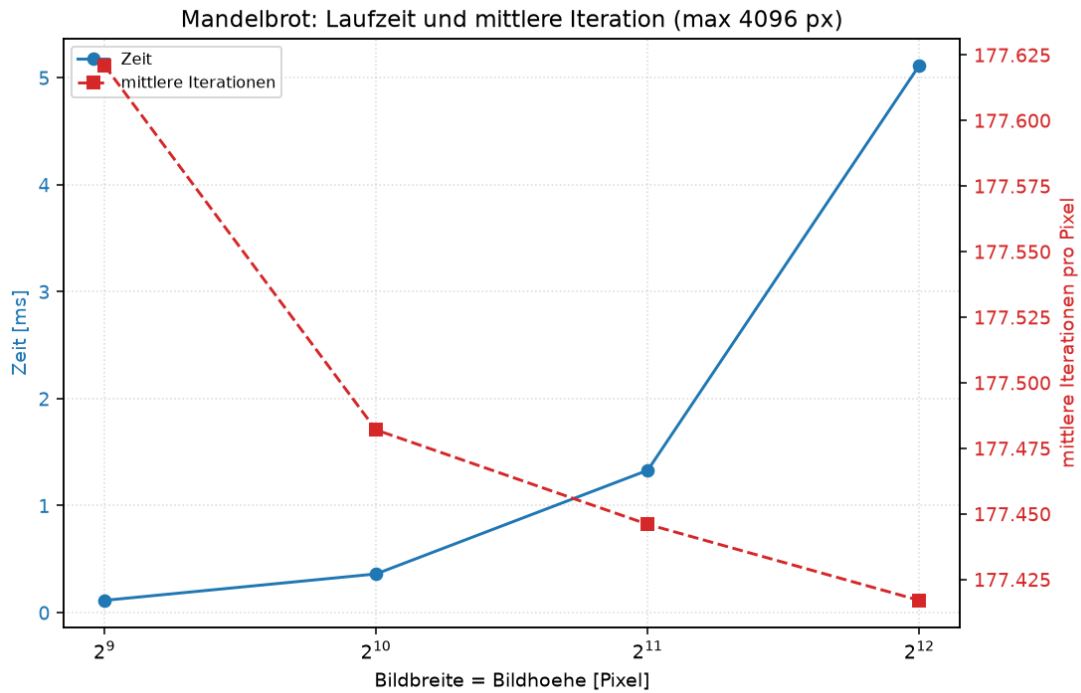


Abbildung 3: Mandelbrot-Sweep: Laufzeit und mittlere Iterationszahl pro Pixel.

Auflösung	Pixel	Zeit [ms]	GFLOP/s, obere Schranke	mittl. Iter.
512^2	262 144	0,114	22 939	177,6
1024^2	1 048 576	0,361	29 008	177,5
2048^2	4 194 304	1,330	31 527	177,4
4096^2	16 777 216	5,115	32 797	177,4

Tabelle 3: Mandelbrot-Messung mit `maxIter = 1000`. Die GFLOP/s-Spalte ist eine obere Schranke, da sie für jedes Pixel die maximale Iterationszahl annimmt.

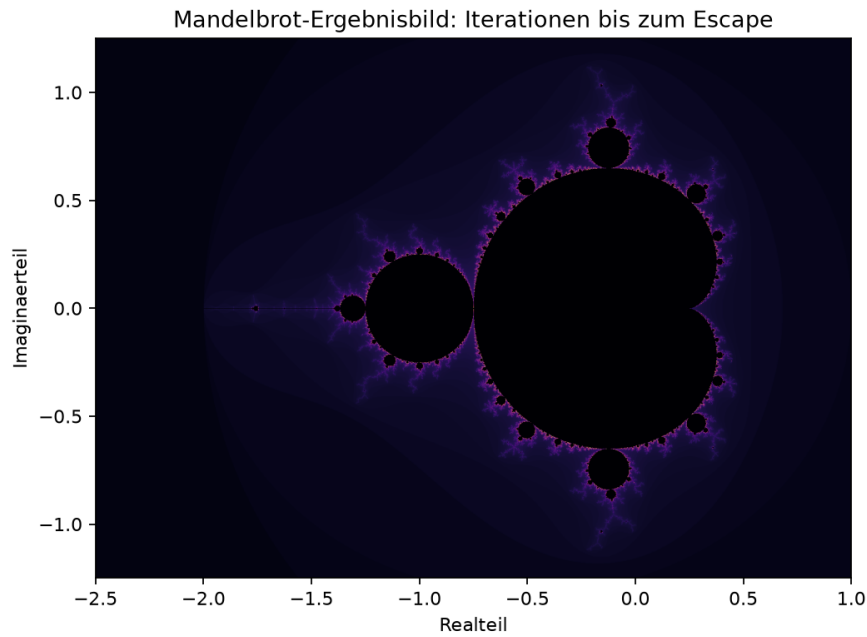


Abbildung 4: Mandelbrot: Die Farbe kodiert die Anzahl der Iterationen bis zum Abbruch.

Die Messung zeigt, dass die mittlere Iterationszahl über die skalierten Auflösungen nahezu konstant bleibt, während die Laufzeit mit der Pixelzahl wächst. Das Bild illustriert die Ursache der Divergenz sichtbar: benachbarte Pixel können sehr unterschiedliche Iterationszahlen besitzen, insbesondere am Rand der Menge.

1.6.2 Monte-Carlo-Kernel

Der Monte-Carlo-Kernel schätzt π , indem zufällige Punkte im Einheitsquadrat geraten werden. Ein Punkt zählt als Treffer, wenn $x^2 + y^2 \leq 1$ gilt. Die Divergenz hängt in diesem Fall direkt von den zufällig gezogenen Zahlen ab.

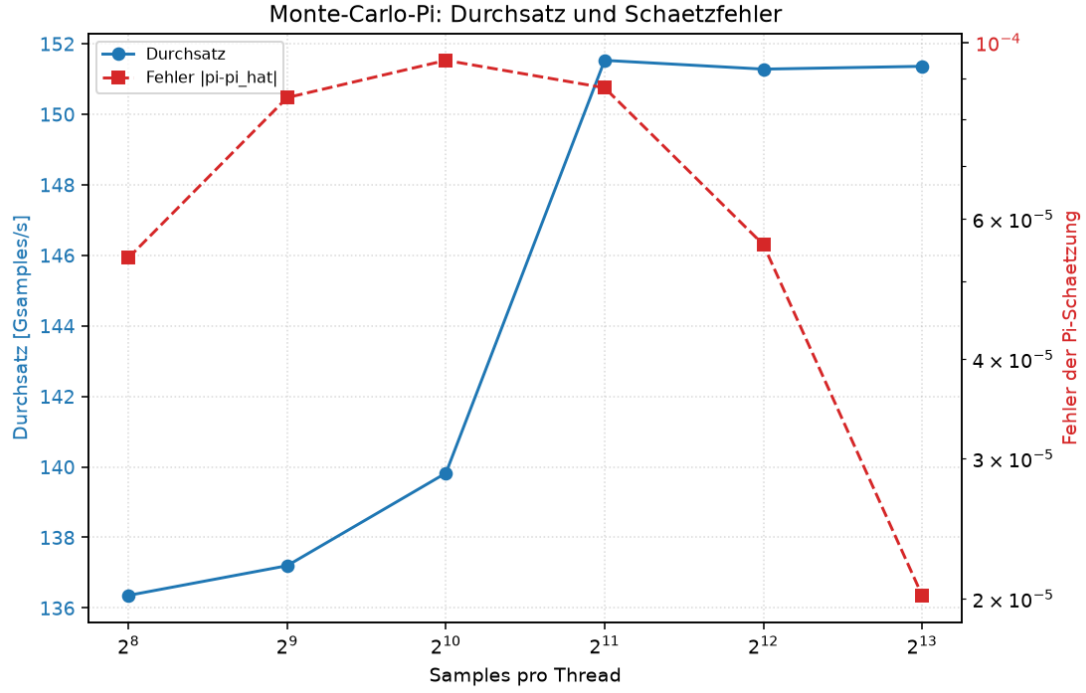


Abbildung 5: Monte-Carlo-Sweep: Stichprobendurchsatz und Fehler der π -Schätzung.

Samp./Th.	Samples [Mrd.]	Zeit [ms]	Gsamp./s	$\hat{\pi}$	Fehler
256	0,268	1,969	136,36	3,141646	$5,36 \cdot 10^{-5}$
512	0,537	3,913	137,20	3,141678	$8,52 \cdot 10^{-5}$
1024	1,074	7,679	139,82	3,141688	$9,49 \cdot 10^{-5}$
2048	2,147	14,172	151,53	3,141680	$8,76 \cdot 10^{-5}$
4096	4,295	28,391	151,28	3,141648	$5,57 \cdot 10^{-5}$
8192	8,590	56,752	151,36	3,141613	$2,02 \cdot 10^{-5}$

Tabelle 4: Monte-Carlo-Messung mit 2^{20} Threads.

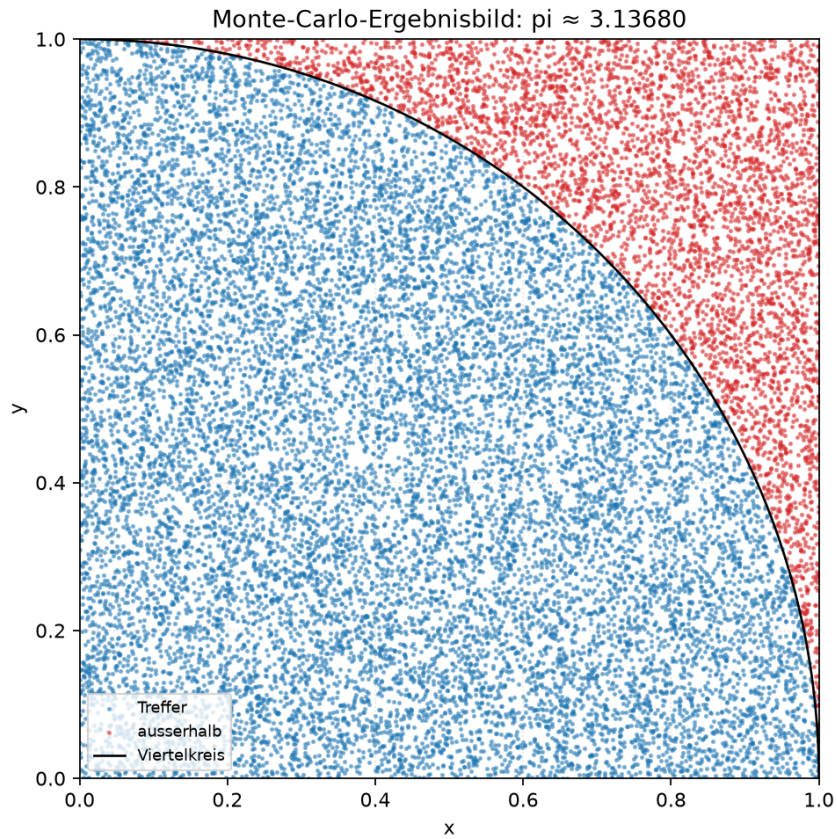


Abbildung 6: Monte-Carlo-Ergebnisbild: zufällige Punkte im Einheitsquadrat, getrennt nach Treffern innerhalb des Viertelkreises und Punkten außerhalb.

Die Tabelle zeigt, dass der Durchsatz ab größeren Stichprobenzahlen pro Thread bei etwa 151 Gsamples/s liegt. Der Fehler der π -Schätzung sinkt nicht strikt monoton, da die Messung stochastisch ist; mit wachsender Gesamtzahl an Samples wird die Schätzung aber tendenziell stabiler.